

Contents

conway	1
1. The gap	1
2. The thesis	1
3. The distinctive capabilities	2
4. The context-curation harness, realized: the interactive surface	5
5. Who it's for	6
6. The commitments	6

conway

The case for conway — the distinctive capabilities it brings to agentic coding that other harnesses do not.

1. The gap

Most agentic-coding harnesses treat **context as an afterthought and agents as flat**. The consequences are not theoretical. The MAST study — “*Why Do Multi-Agent LLM Systems Fail?*” (Cemri et al., UC Berkeley, 2025; arXiv:2503.13657) — annotated 1,600+ execution traces across seven popular multi-agent frameworks and measured end-to-end failure rates of **41–87%**. The two dominant failure categories are structural:

- **Specification / system-design failures (41.8%)** — lost conversation history, step repetition, termination failures.
- **Inter-agent misalignment (36.9%)** — context resets, information withholding, task derailment.

Both are the same root cause wearing different masks: **context flow between agents is opaque**. You cannot see what a child knew, where its context came from, or why it drifted. When the harness hides that, the agent — and the operator — are flying blind.

Meanwhile, on the single-agent side, the context window is treated as a bottomless buffer. When it fills, the harness answers with invisible truncation or automatic compaction. Routing across local and cloud backends is a black box: a request fails, and you cannot tell which model served it, why it was chosen, or why it broke. “Subagents” are one blurry feature with a partial-inheritance knob, where the knob’s semantics are nobody’s idea of a primitive.

conway’s bet is that these are all the same problem — **context is the scarce resource, and the harness should treat it that way, visibly and steerably**.

2. The thesis

conway is a Rust agent harness for agentic coding. Its differentiator is **granular, first-class agent primitives** — fork and spawn as distinct, composable building blocks — combined with **deliberate, inspectable context economy**. The same harness drives an interactive terminal UI, an embeddable Rust library, and a scriptable one-shot mode.

The argument of this paper is that conway brings nine capabilities to the table that, taken together, no other harness in its class offers. Each is a response to a specific, observable failure of the status quo.

3. The distinctive capabilities

3.1 Fork and spawn — granular, first-class agent primitives

The common approach. A single “subagent” feature with a partial-inheritance knob — some context flows to the child, some doesn’t, and the rule is an implementation detail. The knob is not a primitive; it is a leaky abstraction that nobody reasons about cleanly.

conway. Two distinct primitives, sharply defined and meant to compose:

- **Fork** — a child inherits the forker’s *entire* effective context at the fork point as a literal, immutable, cache-friendly prefix. Storage is **O(1)**: one header line, zero records copied. Siblings forked at the same point share a single memoized prefix allocation. After the fork the two sessions are independent append-only logs; the inherited prefix is bounded at the fork sequence and is **byte-identical to a snapshot** — prompting the parent never reaches the child, and vice versa.
- **Spawn** — a clean-slate child that requires an agent definition and inherits *no* parent context.

There is **no partial-inheritance knob**. Fork and spawn are genuinely separate primitives, and that separation is what makes them composable: tournaments, adversarial panels, parallel exploration, and the aggregate pattern (fork, then spawn N differently-prompted children, then collect their results) are all expressions of two building blocks rather than one overloaded feature.

Bidirectional steering is a separate, explicit channel: parents steer children (applied at turn boundaries) and can soft- or hard-cancel them; children report progress and terminal results back. A parent’s **await** on a child **can never hang** — the supervisor synthesizes a terminal result on panic, budget exhaustion, or cancellation. Messaging is not context inheritance; the two are orthogonal.

3.2 Full context provenance and a visible agent tree

The common approach. Context flow between agents is invisible — the exact condition the MAST study identifies as the root of inter-agent misalignment.

conway. Every context segment carries typed provenance (a nine-variant model): for any agent, you can inspect what its context contains and where every part of it came from. The agent tree is a **first-class interactive surface**, not a debug afterthought — the `/agents` panel shows every live and finished agent, its recipe label (`fork @seq N, @<agent_def>, (inherit), (ephemeral)`), and its place in the hierarchy, with a draw-time visibility filter (active / all / finished) that never mutates the tree. You can see what a child knew and where it came from because the harness records it and shows it to you.

3.3 Deliberate context economy

The common approach. The context window fills; the harness silently truncates or auto-compacts. The operator finds out when the agent’s behavior degrades, which is well after the

fact and never attributable.

conway. Context is the scarce resource, and the harness treats it that way with explicit, visible mechanisms:

- **Hierarchy** — lean parent agents delegate detail downward, so deeper agents carry small, focused contexts. Context rot is empirically real long before nominal window limits; over-inheritance is a defect on par with under-inheritance.
- **Admission control** — an admission gate **rejects** requests that won't fit, reserving headroom for output and reasoning. Predictable failure over silent truncation or surprising cost.
- **Pluggable per-call curation** — a `ContextHook` port lets a host mask records, edit the system prompt, filter the announced tool set, or react to context overflow. No built-in curation policy ships as the default; the hook is the seam where policy lands. A log-preserving record mask marks records to exclude from LLM calls while keeping them in the append-only log — reversible, not destructive.

The mechanisms are expected to evolve — automated curation may well be warranted as the design space clarifies — but the objective is fixed: *spend context deliberately*.

3.4 Declarative, explainable, capability-aware routing

The common approach. Routing is a black box. A request fails and you cannot tell which model served it, why it was chosen, or which layer broke.

conway. Routing is declarative and explainable end to end:

- **Role→model chains** with explicit, health-filtered fallback. Every response is traceable to *which model served it and why* (**conway routes explain**).
- **Capability-aware adapters** — per-model tool-calling reliability, streaming behavior, and prompt-caching support are first-class, rather than a lowest-common-denominator interface that pretends every backend is the same.
- **Dual circuit breakers** (transport vs. probe) with a background prober and a failover loop that records health observations and fails over on transport, server, and rate-limit errors — without killing your *only* configured endpoint over a single transient blip.
- **Prompt caching is an economics optimization, never correctness-bearing.** Identical results whether caching is available, evicted, or disabled — verified by byte-identity tests.

3.5 One harness, three interfaces — each genuinely good at its job

The common approach. A tool is either an interactive REPL or a library or a CLI, and the other surfaces are leaky projections of the primary one.

conway. One library serves three consumption modes equally, and each is allowed to be genuinely good at its job:

- **Interactive TUI** (the primary surface) — a single-column, copy-paste-friendly conversation stream, a live `/`-command palette, an on-demand agent-tree panel, ephemeral side-questions, and explicit fork/spawn/steer control over running children.
- **Embeddable Rust library** — the same harness as a single dependency: fully async, event-streamed, no process boundary. For host applications (an IDE, a Tauri app, a service) that need inference and agent orchestration in-process.

- **One-shot (-p)** — a well-mannered Unix filter: model output on stdout, diagnostics on stderr, `text|json|jsonl` output, stable exit codes, and fail-closed tool permissions (an empty allow-list denies every tool, since there is no operator to prompt).

No capability is trapped in one surface.

3.6 A durable, inspectable record

The common approach. Session state is ephemeral, or it is persisted in a format that only the harness can read.

conway. Sessions persist durably as an append-only JSONL log per session, and in-memory state is a cache over the persisted record. The discipline is **persist-before-act**. The record survives crashes; sessions are resumable; any persisted session can be forked-from at any sequence, transitively across multi-level fork chains. It is inspectable with ordinary tools (`conway sessions list | show | tree | export`). Curation of what goes to the model is explicit and reversible — the record is the truth, and what the model sees is a steerable view over it.

3.7 Predictability over cleverness

The common approach. The harness is clever, and cleverness fails in surprising ways — silent truncation, infinite tool-call loops, results that look right but aren’t.

conway. Predictability is a design objective, enforced in concrete mechanisms:

- The **admission gate** rejects oversized requests rather than silently truncating.
- **Result-contract schema validation** with a single retry, then an explicit refusal — never a silent bad result.
- **Repeated-step detection** so an agent cannot loop on the same tool call indefinitely.
- **Mandatory hard budgets** (token and deadline), enforced as hard ceilings.
- **Usage errors surface directly** — a caller-chosen session id that collides gets a message directing you to `--resume`, not a silent overwrite.
- **Prompt-cache reuse is never correctness-bearing** (§3.4).

3.8 A small core, extensible by construction

The common approach. The harness is a monolith; “extension” means forking it. Built-in tools are privileged internals.

conway. An eight-crate Cargo workspace with strictly downward dependencies (ports-and-adapters): `conway-core` holds domain types and port traits only — no I/O. Every capability is a plugin. The `SubagentHost` port breaks the tools-runtime cycle so tools can spawn sub-agents without an upward dependency. The built-in tools — filesystem, shell, subagents, reporting — are implemented on the **same public plugin API** that third parties use. Nothing implemented on the **same public plugin API** that third parties use. Nothing is privileged. The plugin contract is small, semver-disciplined, and the stable surface the harness commits to.

The convention this enables is that **things the core could hardcode, it doesn’t**. Consider compaction. Other harnesses ship a built-in compactor — a generic policy that decides what to forget on the user’s behalf, opaque and one-size-fits-all. `conway` has no first-class compaction feature. When a context needs condensing, the core offers the seams and leaves the policy to the operator: a `ContextHook` that masks records from LLM calls (reversible, log-preserving), or a

spawned compaction worker — a child agent forked to summarize a region of the record and hand the result back. Granular control over what gets condensed, by whom, and under what instructions, instead of a generic solution everyone has to live with. The same shape repeats across the harness: the core provides primitives and ports; the policy is yours to write.

The harness’s responsibility ends at the **permission gate**. Sandboxing, worktree isolation, and file-conflict prevention belong to an agent’s own tools, not the core — a deliberate boundary that keeps the core small and the extension surface honest.

3.9 Security and reliability posture

The common approach. A crashed agent takes the session with it; cross-session access is governed by convention.

conway.

- **Cross-session agent access is rejected** (`AgentNotInSession`): an agent handle cannot drive a session it does not belong to.
 - **Permission gate model** — allow-list, deny-all, and interactive-prompt gates, with a callback surface for the embedder. One-shot mode defaults fail-closed.
 - **The supervisor guarantees a terminal result** on panic, budget exhaustion, or cancellation — a parent waiting on a child is never left hanging.
-

4. The context-curation harness, realized: the interactive surface

The capabilities above are not abstractions; they are concretely wired into the interactive surface. The agent tree, the provenance model, and the fork/spawn primitives are what the operator actually touches:

- **/agents is the single agent surface.** Every row shows the agent’s recipe label — `fork @seq N` (with the inherited fork point), `@<agent_def>` (a spawn with a named definition), `(inherit)` (a spawn that inherited the parent’s role/model), or `(ephemeral)` (a throwaway /ask fork). A `v` key cycles a draw-time visibility filter (active-only / all / finished-only) that never mutates the tree. The old `/tree` command is a hidden alias that renders from the same panel — the same nodes, the same labels, unfiltered.
- **/ask is a single-turn modal with three forced fates.** Asking forks an ephemeral child of the asker (visible in `/agents` marked `(ephemeral)`), runs one turn, and opens a modal over the answer. Closing the modal forces exactly one choice: **fork** (promote the child to a persistent session), **pull in** (merge the question and answer into the parent’s own transcript — the question re-stamped `Provenance::MergedAsk`, the assistant records copied verbatim — then purge the child), or **discard** (purge outright). There is **no fourth way out**: quitting with the modal open is the discard fate. This is provenance-preserving by design — the user explicitly chooses whether the answer enters the durable record, and a crashed process leaves only janitorial residue that the next startup sweeps.
- **Natural-language intent on /fork and /spawn, with a mandatory confirmation card.** Free text after `/fork` or `/spawn` is classified by a cheap model (an ephemeral, tool-less, one-turn session under a declarative `intent` role), and the result is shown in a confirmation card **before anything is created** — `[enter]` confirm, `[e]` edit (drop the classified prompt

into the input line), [esc] manual (the raw text, the caller's default recipe). The card is **the trust gate**: inference never silently chooses. The classifier's output is untrusted and strictly validated — a hallucinated agent definition is stripped, an invalid recipe degrades to the verbatim passthrough, and a confused cheap model can never break the command. Explicit @<agent_def> syntax and bare invocations skip inference entirely.

- **conway_ask** — a **model-facing ephemeral-fork tool**. It runs a prompt in an ephemeral fork of the calling agent and returns the child's **full reply text** (not a truncated summary), so the model can compose it into a **conway_subagent** spawn and keep curation and context-drafting inference **out of the orchestrator's own context window**. An optional **tools** arg narrows the child's tool set (`ToolSelector::Only`, narrowing-only) — e.g. `{"prompt": "summarize the diff", "tools": ["read"]}` for read-only inspection. **ask** is a composition of fork, not a third primitive.

This is the point where the thesis becomes tactile: the operator sees the agent tree, sees the provenance, steers the children, and decides — explicitly, every time — what enters the record.

5. Who it's for

- **The interactive power user** driving coding agents from a terminal, who wants the agent tree and context flow visible and steerable rather than hidden.
 - **The automation engineer** who needs the same harness behind a clean one-shot CLI — streaming, structured output, stable exit codes, fail-closed permissions.
 - **The host application** that needs inference and agent orchestration in-process, behind a single Rust dependency, with a flat ordered event stream a UI can render directly.
 - **The extender** building tools or backends on a stable, semver-disciplined plugin contract that the built-ins themselves use.
 - **The operator** who needs explainable routing, predictable failure, and a durable record they can inspect with ordinary tools.
-

6. The commitments

The mechanisms above are expected to evolve — automated context curation, richer steering, broader extension surfaces, editor reach via an Agent Client Protocol adapter as that protocol matures. The commitments expected to survive any realignment are the ones this paper has argued:

1. **Granular, composable agent primitives** — fork and spawn, not one blurry subagent.
2. **Deliberate context economy** — context is the scarce resource, treated visibly and steerably.
3. **Explainable routing** — every response traceable to which model served it and why.
4. **A durable, inspectable record** — persist-before-act; the record is the truth.
5. **A small, extensible core** — capabilities as plugins, built-ins unprivileged.
6. **Interfaces that are each genuinely good at their job** — no feature trapped in one surface.

conway is licensed **AGPL-3.0-only**. Free to distribute and modify as long as source is provided.